Threat Model

Date updated: 2026-06-03

Scope

This threat model covers the backend/database part of the secure messaging project. The backend is an authenticated relay for encrypted direct 1:1 messages. It stores users, password hashes, refresh-session hashes, public device keys, public one-time prekeys, opaque encrypted message payloads, message visibility metadata, blockchain anchor metadata, and audit logs.

The backend does not decrypt message payloads, store plaintext message content, store private keys, store Signal ratchet/session state, perform client cryptography, provide group chat, or submit Sepolia transactions from FastAPI request handlers. Sepolia submission is handled by the separate backend blockchain worker.

Brief Requirements Covered

The Computer Networks and Cybersecurity brief asks for secure connectivity, server-side security, vulnerability testing, architecture documentation, tested backend processing, and controls against improper input validation, broken authentication, broken access control, injection, cryptographic misuse, security misconfiguration, sensitive data exposure, and vulnerable components.

Backend evidence for these requirements is:

- Secure connectivity and headers: `backend/app/main.py`, `backend/deploy/nginx/epic-messaging-api.conf`, `backend/scripts/check_tls_connection.py`, `backend/tests/integration/test_security_headers.py`.
- Authentication: `backend/app/services/auth_service.py`, `backend/app/services/password_service.py`, `backend/app/services/token_service.py`, `backend/app/api/deps.py`, `backend/tests/security/test_auth_security.py`.
- Access control: `backend/app/services/message_service.py`, `backend/app/repositories/message_repository.py`, `backend/app/services/blockchain_anchor_service.py`, `backend/tests/security/test_access_control_security.py`.
- Input validation and sensitive-data protection: `backend/app/schemas/*.py`, `backend/app/main.py`, `backend/app/services/audit_service.py`, `backend/tests/security/test_input_validation_security.py`, `backend/tests/security/test_sensitive_data_security.py`.
- Database and injection controls: `backend/app/repositories/*.py`, `backend/app/db/session.py`, `backend/tests/security/`

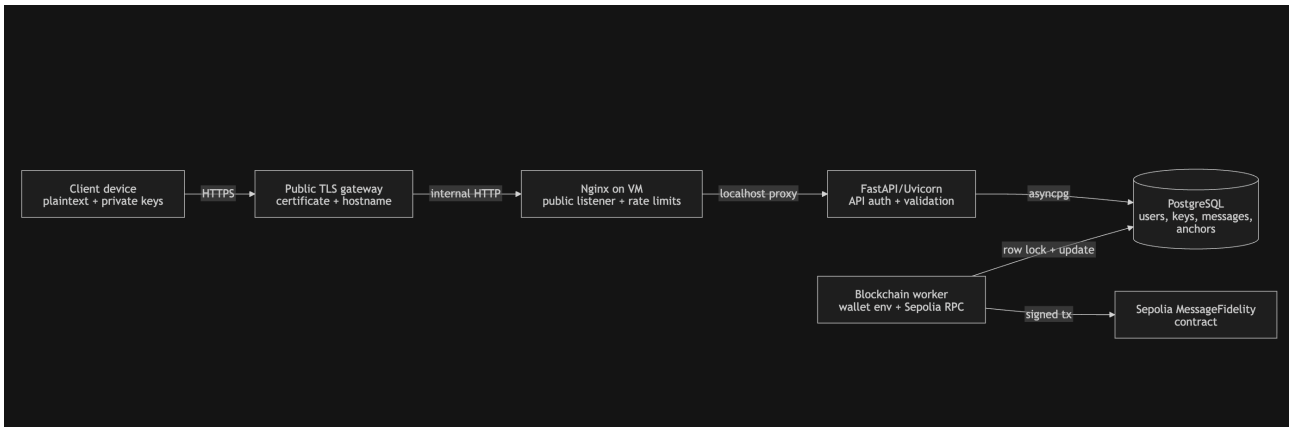
`test_input_validation_security.py`.

- Blockchain fidelity metadata: `backend/app/core/blockchain_hashing.py`, `backend/app/services/blockchain_anchor_service.py`, `backend/app/workers/blockchain_worker.py`, `backend/tests/integration/test_blockchain_routes.py`.
- Final validation evidence: `docs/security/security_test_results.md`.

Assets

Asset	Security property needed	Backend storage
User account identity	Integrity, authenticity	`users` table
Password verifier	Confidentiality against offline cracking	Argon2id PHC hash only
Access token	Authenticity, expiry, replay window control	Client-held JWT; not stored by backend
Refresh token	Confidentiality, replay control	Client receives raw token; DB stores HMAC-SHA256 hash
Public device keys	Integrity and correct ownership	`device_keys` table; public fields only
Public one-time prekeys	Integrity and one-time consumption metadata	`one_time_prekeys` table
Encrypted message payload	Confidentiality of plaintext; integrity of stored ciphertext metadata	`messages.wire_payload_json` as opaque encrypted relay data
Message metadata	Integrity and access control	sender/recipient/device/ timestamp/visibility columns
Blockchain anchor metadata	Integrity and non-repudiation evidence	`blockchain_anchors` table
Audit log	Integrity, limited disclosure	`audit_logs` table with allowlisted details
Deployment secrets	Confidentiality	environment/systemd config, not source
PostgreSQL data	Integrity, availability, limited disclosure	local PostgreSQL service/ container on VM

Trust Boundaries



Important boundary decisions:

- Client devices are trusted to perform end-to-end encryption and protect private keys.
- Public network traffic is protected by TLS at the public gateway.
- FastAPI is not internet-facing directly; Nginx proxies to localhost Uvicorn.
- FastAPI owns authentication, validation, access control, audit logging, and pending anchor creation.
- PostgreSQL is trusted storage but still treated as readable by an honest-but-curious or compromised backend scenario.
- The backend blockchain worker is the only backend component that uses Sepolia RPC, wallet private key, and contract address environment variables.

Attackers

Attacker	Capability
Anonymous attacker	Sends unauthenticated requests to public routes such as register, login, refresh, and health-style API paths.
Passive network attacker	Observes traffic between client and public service endpoint.
Active network attacker	Attempts TLS downgrade, MITM, header manipulation, replay, malformed payloads, or token tampering.
Authenticated malicious user	Holds a valid account and probes other users' messages, keys, anchors, and rate limits.
Malicious sender	Sends malformed encrypted payload metadata, attempts sender spoofing, or abuses forwarding/revocation.
Malicious recipient	Attempts to keep access after revocation, inspect unrelated anchors, or misuse forwarded provenance.
Compromised token holder	Holds a stolen access or refresh token.
Database attacker	Reads or modifies PostgreSQL records.
Honest-but-curious server/operator	Can inspect backend database and logs but follows application code.
Compromised server	Controls FastAPI, environment variables, database access, and responses.
Fully compromised client device	Reads plaintext/private keys on the endpoint and acts as the user.

Attack Surfaces

- Auth routes: `POST /api/v1/auth/register`, `POST /api/v1/auth/login`, `POST /api/v1/auth/refresh`, `POST /api/v1/auth/logout`, `GET /api/v1/auth/me`.
- User discovery route.
- Public device key upload route.
- Public one-time prekey upload route.
- Prekey bundle fetch route.

- Message send, inbox/sent listing, fetch, forward, revoke, delete, and download-style retrieval routes.
- Blockchain anchor create/status/verify routes.
- Pydantic validation and sanitized error handling.
- JWT Bearer parsing and refresh-token rotation.
- Audit logging.
- Rate limiting and Nginx public edge controls.
- CORS and security header configuration.
- PostgreSQL persistence and migrations.
- Backend blockchain worker and Sepolia RPC/contract interaction.

STRIDE Analysis

STRIDE category	Threat	Backend control	Evidence
Spoofing	Attacker forges a JWT or sends another user's sender ID.	JWT signature/expiry/claim/type checks; sender ID is always `current_user.id`; message schema rejects unknown `sender_user_id`.	`token_service.py`, `deps.py`, `schemas/message.py`, auth/access-control tests
Tampering	User modifies someone else's message state or anchor metadata.	Message service checks sender/recipient visibility; revocation is sender-only; anchor lookup requires linked message access.	`message_service.py`, `blockchain_anchor_service.py`, access-control and blockchain route tests
Repudiation	User denies sending or forwarding an encrypted message record.	Audit logs capture message/auth/key events; pending anchors derive Keccak digests from canonical encrypted message metadata including forwarding lineage.	`audit_service.py`, `blockchain_hashing.py`, audit and blockchain tests
Information disclosure	API responses or logs leak passwords, tokens, private keys, plaintext, or ciphertext to unauthorized users.	Response schemas omit password/session hashes; validation errors are sanitized; audit details are allowlisted; message/anchor routes are user-scoped.	`schemas/*.py`, `main.py`, `audit_service.py`, sensitive-data tests
Denial of service	Brute-force login, register spam, prekey scraping, or message spam.	FastAPI fixed-window rate limits and Nginx `limit_req`; payload size limits on schemas.	`deps.py`, `rate_limit.py`, Nginx config, rate-limit tests
Elevation of privilege	Authenticated user reads, deletes, revokes, forwards, or verifies anchors for another user's messages.	`get_current_user()` plus service-layer object checks and safe 404s for inaccessible objects.	`deps.py`, `message_service.py`, `message_repository.py`, access-control tests

Detailed Threats

Passive Network Attacker

Threat: the attacker observes client/backend traffic and tries to learn credentials, tokens, keys, plaintext messages, or message contents.

Controls:

- Public clients connect through `https://kfc.theburkenator.com`.
- TLS protects credentials, access tokens, refresh tokens, public key uploads, encrypted message payloads, and anchor lookups across the public network.
- Backend responses include no plaintext message content because message payloads are client-encrypted before submission.
- Passwords are stored as Argon2id hashes and refresh tokens are stored as HMAC hashes, reducing damage from database disclosure.

Evidence:

- TLS and deployment: `docs/architecture/network_architecture.md`, `backend/scripts/check_tls_connection.py`.
- No plaintext/private-key storage: `backend/app/models/message.py`, `backend/app/schemas/message.py`, `backend/app/schemas/device_key.py`, `backend/tests/security/test_sensitive_data_security.py`.
- Password/token storage: `backend/app/services/password_service.py`, `backend/app/services/token_service.py`.

Residual risk:

- Traffic between the TLS gateway and VM is internal HTTP. Deployment controls keep Uvicorn bound to localhost behind Nginx and document the public boundary.

Active Network Attacker

Threat: the attacker modifies traffic, redirects clients, replays tokens, injects invalid payloads, strips HTTPS, or tampers with forwarded headers.

Controls:

- Clients validate the public certificate and hostname.
- FastAPI can reject non-HTTPS requests with `ENFORCE_HTTPS=true`.
- `TRUST_X_FORWARDED_PROTO=true` uses only the first forwarded-proto value from the configured gateway path.
- HTTPS responses include HSTS when enabled.
- JWT signatures and required claims detect modified access tokens.

- Refresh-token rotation rejects replay after a token is used.
- Pydantic schemas reject malformed or oversized inputs before service logic.

Evidence:

- HTTPS/header middleware: `backend/app/main.py`.
- Production config validation: `backend/app/core/config.py`.
- JWT/refresh checks: `backend/app/services/token_service.py`, `backend/app/services/auth_service.py`.
- Validation tests and header tests: `backend/tests/security/test_input_validation_security.py`, `backend/tests/integration/test_security_headers.py`.
-

Broken Authentication

Threat: attackers guess passwords, enumerate accounts, forge tokens, use refresh-token replay, or authenticate inactive users.

Controls:

- Registration stores Argon2id password hashes.
- Duplicate-user and login failures use generic messages.
- Inactive users fail login and Bearer authentication.
- Access tokens are short-lived JWTs signed with HS256 only and require `sub`, `role`, `jti`, `iat`, `exp`, and `type=access`.
- Refresh tokens are high-entropy opaque values; DB stores HMAC-SHA256 hashes only.
- Refresh rotation locks the active session row, revokes it, and creates a new hash.
- Logout revokes the matching active refresh session without revealing whether it existed.

Evidence:

- `backend/app/services/auth_service.py`
- `backend/app/services/password_service.py`
- `backend/app/services/token_service.py`
- `backend/app/api/deps.py`
- `backend/tests/security/test_auth_security.py`
- `backend/tests/integration/test_auth_routes.py`
- `backend/tests/unit/test_token_service.py`

Residual risk:

- MFA and user-facing session management are not part of the backend prototype.
- A stolen valid access token works until expiry. A stolen refresh token works until rotation or revocation.

Broken Access Control

Threat: authenticated users access messages, anchors, key records, revoke/delete actions, or forwarding provenance for objects they do not own.

Controls:

- Protected routes require `get_current_user()`.
- Message list and fetch queries are scoped to sender/recipient visibility.
- Revocation is sender-only.
- Delete records per-user visibility timestamps.
- Forwarding first checks access to the original message, then creates a new encrypted payload with server-controlled `forwarded_from_message_id`.
- Anchor create/status routes require access to the linked message.
- Inaccessible messages and anchors return safe not-found responses.

Evidence:

- `backend/app/services/message_service.py`
- `backend/app/repositories/message_repository.py`
- `backend/app/services/blockchain_anchor_service.py`
- `backend/tests/security/test_access_control_security.py`
- `backend/tests/integration/test_message_routes.py`
- `backend/tests/integration/test_blockchain_routes.py`

Residual risk:

- Every new route that exposes message, key, or anchor data requires the same object-level checks and regression tests.

Improper Input Validation

Threat: attacker sends invalid UUIDs, negative IDs, unknown fields, oversized payloads, malformed base64 key fields, fake private-key fields, malformed wire payloads, or sensitive values designed to be reflected in errors.

Controls:

- Request schemas inherit strict model behavior and reject unknown fields.

- IDs are UUIDs or positive integers.
- Base64 public key fields are validated.
- `wire_payload_json` has a maximum byte size and libsignal-style structure validation.
- Validation error responses remove submitted input and redact sensitive field names.

Evidence:

- `backend/app/schemas/common.py`
- `backend/app/schemas/message.py`
- `backend/app/schemas/device_key.py`
- `backend/app/main.py`
- `backend/tests/security/test_input_validation_security.py`
- `backend/tests/unit/test_wire_payload_validation.py`

Residual risk:

- Schema limits and validation tests must evolve with the cryptography package's final wire format.

Injection

Threat: attacker submits SQL fragments through username, email, UUID, search, key, message, or anchor fields.

Controls:

- Repository functions use SQLAlchemy ORM query builders and bound values.
- Pydantic validation rejects malformed IDs before repository calls.
- Tests cover injection-style login and request payloads.
- Bandit was run across `backend/app` and `backend/scripts` during the final evidence pass.

Evidence:

- `backend/app/repositories/*.py`
- `backend/tests/security/test_input_validation_security.py`
- `backend/tests/security/test_auth_security.py`
- `docs/security/security_test_results.md`

Residual risk:

- Future raw SQL requires review and tests before submission.

Cryptographic Misuse In Backend Scope

Threat: backend implements weak password hashing, accepts unsafe JWT algorithms, stores raw refresh tokens, hashes blockchain values incorrectly, or tries to perform custom message encryption.

Controls:

- Passwords use Argon2id through `argon2-cffi`.
- JWT decode pins HS256 and requires the access-token claim set.
- Refresh-token storage uses HMAC-SHA256 with a separate deployment secret.
- Message encryption is not implemented by the backend; the backend only relays opaque encrypted payloads.
- Blockchain digests use Ethereum Keccak-256 through `eth_hash.auto.keccak` and canonical JSON ordering.

Evidence:

- `backend/app/services/password_service.py`
- `backend/app/services/token_service.py`
- `backend/app/core/blockchain_hashing.py`
- `backend/tests/unit/test_password_service.py`
- `backend/tests/unit/test_token_service.py`
- `backend/tests/unit/test_blockchain_anchor_service.py`

Residual risk:

- The backend cannot prove that a ciphertext was produced by the intended client cryptographic session or consumed prekey. That proof belongs to the client/cryptography deliverable.

Sensitive Data Exposure

Threat: API responses, validation errors, audit logs, or database rows leak plaintext messages, passwords, token hashes, raw refresh tokens, private keys, or secret configuration.

Controls:

- User response schemas omit password hashes.
- Refresh-session hashes are not returned by API responses.
- User discovery hides email and key material.
- Device key schemas contain public fields only.
- Validation errors do not echo request body values.
- Audit logging stores allowlisted details and omits passwords, tokens, key material, plaintext,

and `wire_payload_json`.

- `.env.example` contains placeholders and real `.env` files are ignored.

Evidence:

- `backend/app/schemas/*.py`
- `backend/app/services/audit_service.py`
- `backend/app/main.py`
- `backend/.env.example`
- `backend/tests/security/test_sensitive_data_security.py`
- `backend/tests/integration/test_audit_logging.py`

Residual risk:

- Message metadata remains visible to the backend and database. This includes sender, recipient, device IDs, timestamps, forwarding lineage, and visibility state.

Blockchain Fidelity Evidence

Threat: a user disputes whether an encrypted message record existed, claims a forwarded record was altered, or claims the backend changed stored encrypted metadata after delivery.

Controls:

- Send and forward create a pending `blockchain_anchors` row inside the same DB transaction as the message row.
- `record_id` is `keccak256("message:" + message_id)`.
- `digest` is Keccak over canonical encrypted message metadata: message ID, sender/recipient IDs, device IDs, created timestamp, `wire_payload_json`, and forwarding lineage.
- Anchor status routes expose proof metadata only to users who can access the linked message.
- Production runs the backend blockchain worker separately with Sepolia RPC URL, worker wallet key, and contract address from environment variables.
- The worker locks pending rows with `FOR UPDATE SKIP LOCKED`, submits `MessageFidelity.storeHash(record_id, contentHash)`, waits for the transaction receipt, and records transaction hash, contract address, and anchored timestamp.

Evidence:

- `backend/app/services/message_service.py`
- `backend/app/services/blockchain_anchor_service.py`
- `backend/app/core/blockchain_hashing.py`

- `backend/app/workers/blockchain_worker.py`
- `backend/deploy/epic-messaging-blockchain-worker.service`
- `backend/tests/integration/test_blockchain_routes.py`

Residual risk:

- During Sepolia, RPC, wallet, gas, or worker outage, anchors remain pending until the worker resumes.

Honest-But-Curious Server Or Operator

Threat: backend/operator reads database rows and logs while the application code remains honest.

Controls:

- Backend stores encrypted payloads rather than plaintext.
- Backend stores public keys rather than private keys.
- Password and refresh-token verifiers are one-way values.
- Audit details avoid raw secrets and message payloads.

Residual risk:

- Metadata remains visible: account IDs, sender/recipient relationships, device IDs, key publication timing, prekey consumption metadata, message timestamps, forwarding lineage, anchor status, and transaction hashes.

Compromised Server Or Database

Threat: attacker controls FastAPI, reads/writes DB rows, changes public key material, steals environment secrets, modifies responses, deletes messages, prevents anchors, or captures future tokens.

Controls that still help:

- Stored message payloads remain ciphertext if client private keys are not also compromised.
- Stored passwords are Argon2id hashes.
- Stored refresh tokens are HMAC hashes.
- Confirmed blockchain transaction hashes give external evidence for previously anchored records.

Controls lost:

- Public key integrity, metadata privacy, availability, token confidentiality for future requests, audit-log trust, and pending-anchor integrity.

Residual risk:

- A compromised server can deny service, publish attacker-controlled public keys, alter unanchored metadata, and capture future tokens. Stronger key transparency and client-side

safety-number verification are outside backend scope.

Fully Compromised Client Device

Threat: attacker controls the user's endpoint.

Backend impact:

- The attacker can read plaintext before encryption and after decryption.
- The attacker can access private keys and local cryptographic state.
- The attacker can act as the user while holding valid tokens.

Backend controls:

- Access tokens expire.
- Refresh sessions can be revoked through logout or service-side bulk revocation.
- Server-side audit logs show account activity.

Residual risk:

- Backend controls do not protect plaintext or private keys on a compromised endpoint.

Final Residual Risk Register

Residual risk	Severity	Final control or boundary
No MFA	Medium	Password hashing, generic failures, rate limits, short access-token lifetime, and refresh rotation are implemented; MFA is outside backend prototype scope.
In-memory FastAPI rate limiter	Low/Medium	Nginx edge limits protect the single-VM deployment; distributed limits are a scale-out requirement.
Gateway-to-VM internal HTTP	Medium	Public TLS gateway protects internet traffic; Uvicorn is local behind Nginx; deployment docs record the boundary.
Metadata visible to backend	Medium	Backend stores no plaintext/ private keys; metadata exposure is documented for interview defence.
Backend cannot prove client ciphertext construction	Medium	Backend validates relay structure and prekey metadata only; cryptographic proof is in the client/crypto deliverable.
Pending anchors during worker/RPC outage	Medium	Production worker runs separately and processes persisted pending rows when Sepolia/RPC/service availability returns.
Compromised server can alter public keys	High	E2EE protects old ciphertext when client keys remain safe; key transparency/safety numbers are future hardening.
Fully compromised client	High	Backend revocation and audit controls remain; endpoint plaintext/private-key compromise is outside backend control.